Git Best Practices for KAIST IAM Group

> Adapted from UMD CGS Git Best Practices by Andy Miller and Nishant Gaonkar (originally from Dillon Walton's brown bag series)

Table of Contents

1. Overview

Why Use Git?

- **Collaborate** with others effectively on code
- **Track changes** to your work over time
- **Maintain multiple versions** of code that can be compared and merged
- **Roll back** to earlier versions if something goes wrong

What is Git?

Git is a **version control system** - a software program that manages versions of your files.

- Works as a "time machine" for going back to earlier versions of your code
- Provides excellent support for different versions (branches) of the same project
- Simplifies concurrent work and merging of changes from multiple collaborators

Before Git: Think of manually managing Excel files like `analysis_v1.xlsx`, `analysis_v2_final.xlsx`, `analysis_v2_final_REAL.xlsx`...

With Git: All versions are tracked automatically, and you can always go back to any previous state.

2. Core Concepts

2.1 Repository (Repo)

A repository is a folder that Git tracks. It contains:

- Your project files
- A hidden `.git` folder (the "mini-filesystem" that stores all version history)

2.2 Local vs Remote

Local	Remote
On your computer	On a server (GitHub, GitLab, etc.)
Only you can see it	Shared with collaborators
Where you do your work	Where you backup and share

2.3 Branch

- A **branch** is an independent line of development
- The default branch is usually called `main` (or `master`)
- Create new branches to work on features without affecting the main code
- Branches can be **merged** back together

```
main:        ●──●──●──●──●
                  \       /
feature:           ●──●
```

2.4 Commit

- A **commit** is a snapshot of your code at a specific point in time
- Each commit has a unique ID and a message describing the changes
- Commits are the building blocks of your version history

2.5 Staging Area

Before committing, files must be **staged** (added to the staging area):

```
Working Directory  →  Staging Area  →  Repository
    (your files)        (git add)       (git commit)
```

---

3. Basic Commands

3.1 git init / git clone

Start a new repository:

```
git init
```

Clone an existing repository:

```
git clone https://github.com/kaist-iam/your-repo.git
```

3.2 git status

Check the current state of your repository:

```
git status
```

This shows:

- Which branch you're on
- Which files have been modified
- Which files are staged for commit

3.3 git add + git commit

Stage specific files:

```
git add filename1.R filename2.R
```

Stage all changes:

```
git add .
```

Commit staged changes:

```
git commit -m "Add scenario analysis for Korea region"
```

Commit message tips:

- Complete this sentence: "If I make this commit, it will ___"
- Be specific: `"Update Korea emissions factors"` not `"Update stuff"`

3.4 .gitignore (Important for GCAM/IAM Work)

The `.gitignore` file tells Git which files to **NOT track**. This is crucial for IAM research where output files can be very large.

Create a `.gitignore` file in your repository root:

```

```

```
# GCAM Output Files
output/
*_output/
*.dat
*.basex

# R Files
.Rhistory
.RData
.Rproj.user/
*.Rproj

# Large Data Files
*.csv
!input/*.csv  # Exception: track input CSV files

# Quarto
.quarto/

# System Files
.DS_Store
Thumbs.db

# IDE
.idea/
.vscode/

# Temporary Files
*.tmp
*.log
nul
```

Check if a file is being ignored:

```
git check-ignore -v filename
```

Important: Never commit GCAM output files, database files, or large datasets to Git. They can make the repository extremely large and difficult to work with.

3.5 git branch

List all branches:

```
git branch        # local branches
git branch -a     # all branches (including remote)
```

Create and switch to a new branch:

```
git checkout -b jiheun_main
```

Switch to an existing branch:

```
git checkout main
```

3.6 git merge

Merge changes from one branch into another:

```
# First, switch to the branch you want to merge INTO
git checkout main

# Then merge the other branch
git merge jiheun_main
```

3.7 git pull / git push

Pull changes from remote:

```
git pull origin main
```

Push your changes to remote:

```
git push origin main
```

Understanding pull: `git pull` = `git fetch` + `git merge`

If you want more control, you can do these separately:

```
git fetch origin main          # Download changes
git merge origin/main          # Merge them in
```

---

4. Best Practices

4.1 General Workflow Rules

1. **Always create a new branch for your work**

- Name it `yourname_branchname`, e.g., `jiheun_korea_scenario`
- Never work directly on `main`

2. **Commit frequently**
   - Commit after each notable change
   - Small, frequent commits are better than large, rare ones
3. **Pull before you push**
   - Always pull the latest changes before pushing
   - This reduces merge conflicts
4. **Write clear commit messages**
   - Describe what the commit does, not what you did
   - Good: `"Add carbon tax scenario for Korea"`
   - Bad: `"Updated files"`
5. **Don't commit data files**
   - Use `.gitignore` to exclude output files
   - Keep your repository lightweight
6. **Pull frequently**
   - Sync with remote often to stay up to date
   - This makes merging easier

4.2 Branch Naming Convention

```
yourname_purpose
```

Examples:

- `jiheun_main` - Your personal development branch
- `jiheun_korea_emissions` - Working on Korea emissions analysis
- `jiheun_fix_transport` - Fixing transport sector bug

4.3 Git LFS (Large File Storage)

If you absolutely must track large files (>100MB), use Git LFS:

```
# Install Git LFS (one time)
git lfs install

# Track large file types
git lfs track "*.nc"
git lfs track "*.hdf5"
```

```
# Make sure .gitattributes is tracked
git add .gitattributes
```

However, the best practice is to NOT put large files in Git at all. Instead:

- Store large data files on shared drives or cloud storage
- Document where to find the data in your README
- Only track code and small configuration files in Git

---

5. Git in Practice

5.1 Workflow 1: Getting Updates from Others

When collaborators have pushed changes and you need to get them:

```
# 1. Save all your local files

# 2. Check for updates
git fetch
git log --oneline your_branch..origin/main

# 3. Switch to main branch
git checkout main

# 4. Pull latest changes
git pull origin main

# 5. Switch back to your branch
git checkout jiheun_main

# 6. Merge main into your branch
git merge main

# 7. Resolve any conflicts (see Troubleshooting section)
```

5.2 Workflow 2: Contributing Your Changes

When you want to share your changes with others:

```
# 1. Save all files locally

# 2. Check status
git status

# 3. Stage your changes
git add filename1.R filename2.R
# Or: git add .
```

```
# 4. Commit with a clear message
git commit -m "Add Korea region carbon pricing scenario"

# 5. Switch to main branch
git checkout main

# 6. Pull latest changes
git pull origin main

# 7. Merge your branch into main
git merge jiheun_main

# 8. Resolve any conflicts if needed

# 9. Push to remote
git push origin main

# 10. Switch back to your branch
git checkout jiheun_main

# 11. Merge main back to keep your branch updated
git merge main
```

5.3 Cloning a Repository (First Time Setup)

```
# 1. Navigate to your desired directory
cd C:/GCAM/projects

# 2. Clone the repository
git clone https://github.com/kaist-iam/your-project.git

# 3. Enter the repository
cd your-project

# 4. Create your personal branch
git checkout -b jiheun_main

# 5. Start working!
```

---

6. Troubleshooting

6.1 Resolving Merge Conflicts

When Git can't automatically merge changes:

```
<<<<<<< HEAD
Your version of the code
=======
```

```
Their version of the code
>>>>>>> branch_name
```

To resolve:

1. Open the conflicted file
2. Decide which version to keep (or combine them)
3. Remove the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`)
4. Save the file
5. Stage and commit:

```
git add conflicted_file.R
git commit -m "Resolve merge conflict in scenario analysis"
```

In VS Code: The editor highlights conflicts and provides buttons to accept either version.

6.2 Undo Last Commit (Not Pushed)

```
# Keep changes, just undo the commit
git reset --soft HEAD~1

# Discard the commit and changes
git reset --hard HEAD~1
```

6.3 Discard Local Changes

```
# Discard changes in one file
git checkout -- filename.R

# Discard all local changes
git checkout -- .
```

6.4 Stash Changes Temporarily

When you need to switch branches but have uncommitted changes:

```
# Save changes temporarily
git stash

# Do your other work...
git checkout other_branch

# Come back and restore changes
```

```
git checkout your_branch
git stash pop
```

6.5 View Commit History

```
# Simple log
git log --oneline

# Detailed log
git log

# Log with graph
git log --oneline --graph --all
```

6.6 Accidentally Committed Wrong Files

If you committed files that should be ignored:

```
# Remove from Git but keep the file locally
git rm --cached filename.dat

# Add to .gitignore
echo "filename.dat" >> .gitignore

# Commit the changes
git add .gitignore
git commit -m "Remove accidentally committed data file"
```

6.7 Common Error: "Your branch is behind"

```
# Pull the latest changes first
git pull origin main

# Then push your changes
git push origin main
```

---

7. VS Code Setup

VS Code provides excellent Git integration that can complement terminal usage.

7.1 Recommended Extensions

1. **GitLens** - Enhanced Git capabilities

    - View commit history inline
    - Compare branches
    - Visualize repository graph

2. **Git Graph** - Visual branch/commit graph

7.2 Installing Extensions

1. Open VS Code
2. Press `Ctrl+Shift+X` (or `Cmd+Shift+X` on Mac)
3. Search for "GitLens" and install
4. Search for "Git Graph" and install

7.3 Using Source Control Panel

The **Source Control** panel (left sidebar, looks like a branch icon):

- **Changes**: Shows modified files
- **Staged Changes**: Files ready to commit
- **Click +**: Stage a file (same as `git add`)
- **Click -**: Unstage a file
- **Message box**: Write commit message
- **Checkmark**: Commit (same as `git commit`)

7.4 Using GitLens Graph

1. Click the GitLens icon in the sidebar
2. Click "Commits" to see history
3. Click "Branches" to see all branches
4. Use the Graph view to visualize branch history

7.5 Resolving Conflicts in VS Code

When a merge conflict occurs, VS Code shows:

- **Accept Current Change**: Keep your version
- **Accept Incoming Change**: Keep their version
- **Accept Both Changes**: Keep both
- **Compare Changes**: See side-by-side diff

This is often easier than manually editing conflict markers.

7.6 Terminal vs GUI

Use Terminal for	Use VS Code GUI for
Complex operations	Viewing changes
Scripts/automation	Staging specific lines
When GUI is unclear	Resolving conflicts

Use Terminal for	Use VS Code GUI for
Learning Git deeply	Quick commits

Recommendation: Learn terminal commands first, then use VS Code GUI to speed up common tasks.

---

8. Additional Resources

Official Documentation

- Git Official Website
- Pro Git Book (free)
- Git Reference Manual

Video Tutorials

- FreeCodeCamp Git Crash Course (1 hour)
- Traversy Media Git Crash Course (30 min)
- Corey Schafer Git Tutorial (30 min)

Interactive Learning

- GitHub Skills
- Learn Git Branching - Interactive visualization

Quick Reference

- Atlassian Git Tutorials
- GitHub Git Cheat Sheet

AI Assistants

- ChatGPT / Claude - Great for explaining Git concepts and debugging issues

---

Quick Reference Card

Command	Description
`git status`	Check current state
`git add .`	Stage all changes
`git commit -m "message"`	Commit with message
`git push origin main`	Push to remote
`git pull origin main`	Pull from remote
`git checkout -b name`	Create & switch to new branch
`git checkout name`	Switch to existing branch

Command	Description
`git merge name`	Merge branch into current
`git log --oneline`	View commit history
`git stash`	Temporarily save changes
`git stash pop`	Restore stashed changes

Prerequisites

Before using this guide, ensure you have:

1. **Git installed**
   - Windows: Git for Windows
   - Mac: `brew install git` or Xcode Command Line Tools
2. **VS Code installed** (recommended)
   - Download VS Code
3. **GitHub account**
   - Sign up at GitHub
4. **Git configured:**

```
git config --global user.name "Your Name"
git config --global user.email "your.email@kaist.ac.kr"
```

Last updated: January 2026

KAIST IAM Group - https://kaist-iam.github.io/group/